WHITEPAPER / PROPOSED SPECIFICATION

Agentic Architecture Blueprint (AAB) 0.1

A Proposal for Reviewable Architecture Contracts and an Independent Reference Validator

Human-readable intent. Machine-checkable rules. Independent evidence.

Technical Whitepaper — Discussion Draft

12 October 2026 • Version 0.1 proposal

A vendor-neutral approach to making architectural intent understandable to people, interpretable by AI coding agents, and independently verifiable throughout software delivery.

Contents

Executive summary 3

1. Introduction: from a drawing to a dependable blueprint 3

2. Problem statement and rationale 4
2.1 Four recurring failure modes . . . 4
2.2 Why existing practice is necessary but insufficient . . . 4

3. AAB principles and boundaries 5

4. The proposed architecture model 5
4.1 Preserve 4+1; add Contract and Evidence . . . 5
4.2 Proposed package structure . . . 6
4.3 Minimal data model . . . 6
4.4 Normative language and initial rule types . . . 7

5. A worked example: protecting an architectural boundary 7

6. Architecture governance: two distinct cycles 8
6.1 Design and approval cycle . . . 8
6.2 Governed implementation cycle . . . 8

7. Reference validator: architecture and interface 8
7.1 Validator pipeline . . . 8
7.2 Proposed command surface . . . 9
7.3 The evidence contract . . . 9
7.4 Security and trust boundaries . . . 9

8. Profiles and adoption strategy 10

9. Related work and differentiation 10

10. Implementation roadmap and research validation 11

11. Limitations and open questions 12

12. Conclusion 12

References 13

Status and scope. AAB 0.1 is a **proposed specification and reference-validator design**, not an existing ISO standard, deployed product, or independently validated assurance system. The data model, commands, examples, and conformance profiles in this paper are design proposals for a first implementation. Compatibility with ISO/IEC/IEEE 42010 is an intended design goal, not a certification claim.

Executive summary

AI coding agents can now translate a short request into substantial amounts of working software. Their speed is valuable, but it changes how architectural decisions are made. Choices about module boundaries, dependencies, persistence, security, error handling, and deployment may be introduced during implementation without deliberate architectural review. A system can therefore pass its functional tests while diverging from the architecture its stakeholders intended.

Software engineering already has effective ways to describe architecture: Kruchten's *4+1* views, ISO/IEC/IEEE 42010, architecture decision records, and models-as-code. Newer spec-driven development tools organise requirements, plans, and coding tasks. Architectural fitness functions and policy engines can enforce particular constraints. The gap is not the absence of any single tool. It is the absence of a widely adopted, interoperable contract connecting **approved architectural intent** to **machine-checkable constraints**, **agent instructions**, and **independently produced evidence** of compliance [1–4, 7–11].

This paper proposes the **Agentic Architecture Blueprint (AAB)**: a vendor-neutral, versioned architecture package with three complementary forms. A human-readable account explains the rationale and trade-offs; a structured model identifies architectural elements and relationships; and explicit contracts state which implementation properties must hold. A separate verifier evaluates supported rules and reports both what passed and what could not be established.

AAB introduces two coupled but distinct cycles. In the **design cycle**, humans and AI reviewers refine the blueprint; designated human decision-makers approve a baseline. In the **implementation cycle**, coding agents may propose and implement changes only within that baseline. If implementation calls for a different architecture, the agent proposes an amendment rather than quietly changing the rules. Protected source-control processes and trusted CI—not compliance with a prompt alone—enforce the approval boundary.

The initial AAB 0.1 scope is intentionally modest: a minimal data model, stable identifiers, a small rule vocabulary, an approval-baseline mechanism, a deterministic validator contract, and a standard evidence report. It should work alongside tools such as Spec Kit and OpenSpec, not replace them. Its usefulness must ultimately be tested by whether independent coding agents and validators can interpret the same architecture consistently, detect real violations, and avoid excessive governance overhead.

Central proposition: architecture should evolve from *documentation that developers are encouraged to follow* into a **reviewed, versioned contract whose implementable rules have explicit verification evidence**.

1. Introduction: from a drawing to a dependable blueprint

Consider the blueprint for a building. A drawing is useful because it allows an architect, engineer, and contractor to discuss the same design. But the drawing alone is insufficient: dimensions, structural requirements, materials, approvals, and inspections establish whether the resulting building conforms to the design.

Software architecture has a similar challenge. A diagram might show that a web API talks to a domain service and that the domain service uses a repository interface. Developers can read this as an architectural intention. Yet a coding agent asked to deliver a feature quickly might connect the API directly to a database adapter. The application may run correctly, and an end-to-end test may pass, while an important separation of responsibilities has been lost.

This is not necessarily an AI-specific defect. Humans also introduce architectural drift. What changes with agentic coding is the speed and volume of design decisions made in the course of code generation, often through tools whose temporary conversational context is not a reliable record of long-lived architectural intent. Recent research explicitly discusses implicit decisions made by coding agents and the need for their governance [14].

The proposed AAB approach combines two things that software engineering has historically treated separately: **explaining the intended architecture** and **demonstrating conformance with it**. AAB retains the clarity of familiar views while supplying structured rules that tools can examine and reviewers can audit.

For a non-specialist stakeholder, the core idea is simple: **people approve the design; agents implement it; independent checks verify the parts that can be checked; departures return for approval**.

2. Problem statement and rationale

2.1 Four recurring failure modes

Ambiguous intent. Natural-language architecture descriptions commonly leave critical words—such as "service", "isolated", "secure", or "independent"—without operational definitions. Two developers, or two agents, can make different implementations while believing they followed the same instruction.

Fragmented sources of truth. Architectural diagrams, ADRs, task plans, source-code conventions, CI policies, and agent instruction files frequently evolve independently. A rule can change in one location while an obsolete interpretation survives in another. The problem is not simply documentation drift; it is unclear authority when artifacts conflict.

Unreviewed architecture changes. Spec-driven workflows improve planning, but an implementation tool can still add a dependency, change a trust boundary, or alter persistence semantics without triggering explicit architecture approval. Conversely, a model may be frozen so rigidly that necessary architectural changes become difficult to propose and evaluate.

Weak assurance claims. A successful test run shows that a specified set of tests passed under specified conditions. It does not establish compliance with every architectural principle. Some constraints can be checked statically; others require integration tests, configuration inspection, load tests, manual review, or stronger forms of analysis. Treating "no detected failure" as "architecture verified" creates false confidence.

2.2 Why existing practice is necessary but insufficient

Kruchten's 4+1 model has long shown the value of multiple views for communicating architecture to different stakeholders [1]. ISO/IEC/IEEE 42010 standardises the concepts and expectations associated with architecture descriptions [2]. Architecture decision records preserve the reasoning behind important choices [5]. Models-as-code improve consistency and version control [6]. Specification-driven systems give coding agents more durable requirements and plans than a chat transcript can provide [7–9]. Architecture tests and policy engines can reject known violations [10–11].

AAB builds on these strengths. Its proposed contribution is a **common binding layer**: a precise connection from architectural elements and decisions to scoped rules, applicable code or configuration, independent checks, exception authority, and recorded outcomes. It is designed as an **architecture-description and conformance profile**, not as a competing full software-development lifecycle.

3. AAB principles and boundaries

The 0.1 proposal is guided by seven principles:

1. **Human authority over architectural intent.** AI tools may analyse, challenge, and propose changes, but they do not approve their own changes to binding architectural constraints.
2. **One authoritative structured model.** Element identifiers, relationships, and rule references have a canonical home; prose and diagrams may elaborate on them but should not quietly contradict them.
3. **Explainability for stakeholders.** Every binding rule is accompanied by a statement of intent and rationale that a human reviewer can understand.
4. **Independent enforcement.** A coding agent's self-assessment is not proof. Verification must occur in a separately controlled execution context.
5. **Evidence with explicit limits.** Reports distinguish verified properties, detected violations, unsupported checks, and failures of the checking infrastructure.
6. **Evolution through approved amendments.** Architectural changes are legitimate, but must be identified, reviewed, versioned, and evaluated for impact.
7. **Vendor neutrality and incremental adoption.** Teams can retain their preferred architecture notation, programming languages, CI environment, and coding agents.

Non-goals of AAB 0.1 include replacing ISO 42010, prescribing an implementation methodology, solving arbitrary program verification, guaranteeing defect-free software, standardising prompts, or providing a universal architecture inference engine. AAB must also avoid equating the absence of a checker with a passed check.

4. The proposed architecture model

4.1 Preserve 4+1; add Contract and Evidence

AAB retains the familiar **logical**, **process**, **development**, and **physical** views, tied together by **scenarios**. Each view serves a distinct audience and question [1]:

View	Principal question	Illustrative elements
Logical	What are the system's responsibilities?	Components, services, data ownership
Process	How does the system behave at runtime?	Workflows, concurrency, failure handling
Development	How is implementation organised?	Packages, modules, dependency directions
Physical	Where does the software run and who trusts whom?	Networks, deployment nodes, security boundaries
+1 Scenarios	How do these parts satisfy important use cases?	Happy paths, degraded operation, misuse, recovery

Two **cross-cutting layers** are added, rather than two replacement views:

- **Architecture Contract.** Normative constraints tied to stable model elements, with applicability, rationale, enforcement expectations, and approval policy.
- **Conformance Evidence.** Reproducible records of what was checked, by which implementation, against which blueprint baseline and source revision, and with what result.

This separation matters. Diagrams and narrative explain design choices; formalised contracts specify implementable boundaries; evidence records what verifiers actually observed. AAB should use ISO/IEC/IEEE 42010 concepts such as viewpoints, concerns, and correspondence without claiming that merely adopting AAB establishes ISO conformance [2]. ISO/IEC 25010 supplies an existing vocabulary for product-quality concerns [3].

4.2 Proposed package structure

AAB 0.1 defines a **logical package** rather than mandating a repository layout. One conventional file representation is:

```
architecture/
  blueprint.yaml          # Model, references, profile, and baseline metadata
  views/                  # Human-readable 4+1 descriptions
  decisions/              # Architecture decision records (ADRs)
  contracts/              # Normative, typed rules
  verification/           # Rule-to-check mappings and coverage policies
  amendments/             # Proposed changes and dispositions
  evidence/               # Generated reports; may instead live in CI artifacts
AGENTS.md                 # Optional generated agent-facing summary
```

A source-control repository may store the files; the approved baseline is defined by a content digest over the canonical package contents, not by a mutable branch name alone. The exact canonicalisation algorithm and set of files included in that digest must be normative in the eventual technical specification. AAB 0.1 should select a deterministic procedure and include cross-language test vectors before two implementations claim interoperability.

4.3 Minimal data model

The canonical model should contain the following entity types:

Entity	Minimum meaning
`blueprint`	Identity, schema version, system scope, ownership, references
`element`	Stable ID, type, name, responsibility, source-code/deployment mapping
`relationship`	Source, target, typed relation, optional view attribution
`decision`	ADR reference, rationale, status, affected elements
`constraint`	ID, statement, rule type, scope, severity, rationale, verification requirement
`scenario`	Conditions, action, expected observable outcome, linked concerns
`baseline`	Approved package digest, version, authority, approval timestamp
`evidence`	Rule ID, baseline digest, checked revision, checker ID, finding, coverage

Stable identities are more important than filenames. Renaming a component should not silently create a new rule subject or disconnect existing evidence. Deleted or renamed identifiers need explicit migration rules. Unknown model fields should be handled according to a documented extension policy rather than silently altering rule meaning.

4.4 Normative language and initial rule types

The final specification should use **MUST**, **SHOULD**, and **MAY** with the RFC 2119 / RFC 8174 interpretation for normative requirements [12]. Because this paper is a discussion draft, the requirements below are **proposed obligations**, not externally ratified standards.

The first rule vocabulary should be small: `forbidden_dependency`, `required_dependency`, `restricted_runtime_access`, `configuration_policy`, and `scenario_evidence`. Each type needs a defined semantic domain and supported verification adapter. A rule whose target cannot be resolved is **unverified**, not successful. Free-text principles remain useful context, but cannot automatically become enforceable merely by labelling them mandatory.

5. A worked example: protecting an architectural boundary

Suppose a system adopts three components: `api`, `domain`, and `persistence_adapter`. The intended dependency path is `api → domain → persistence_adapter`. AAB makes the restriction explicit:

```yaml
schema_version: "0.1-proposal"
blueprint_id: example-order-service
components:
  - id: api
    paths: ["src/api/**"]
  - id: domain
    paths: ["src/domain/**"]
  - id: persistence_adapter
    paths: ["src/adapters/persistence/**"]
constraints:
  - id: ARCH-001
    title: No direct API-to-persistence dependency
    statement: >
      API modules MUST NOT directly import persistence adapters.
    type: forbidden_dependency
    source: api
    target: persistence_adapter
    relation: imports
    decision: ADR-0001
    enforcement: blocking
    verification: static_import_graph
```

Illustrative syntax only: this document proposes the schema; it does not claim this YAML is already accepted by a released AAB validator.

An AI agent reading this rule knows that it should introduce a domain-facing interface rather than importing a concrete database adapter into an API module. During review, an independent checker can resolve the configured source paths and inspect imports. If it discovers a forbidden import, `ARCH-001` is violated and a protected CI gate may reject the change.

Importantly, passing this check establishes a **bounded claim**: no forbidden edge was found in the dependency graph constructed by the checker for the specified source scope. It does **not** prove the absence of indirect persistence access via runtime reflection, network calls, dynamic plugins, or unexamined generated code. Those paths require additional rules or different evidence. This distinction is central to the credibility of AAB.

6. Architecture governance: two distinct cycles

6.1 Design and approval cycle

First, stakeholders describe the problem, requirements, operating assumptions, quality concerns, and significant scenarios. An architect—and optionally one or more AI reviewers—develops candidate views and decisions. Reviews should explicitly search for contradictions between views, missing trust boundaries, non-testable claims, failure scenarios, and costs of proposed constraints.

A designated human authority records the disposition of each significant comment: **accept**, **revise**, **reject with rationale**, or **defer**. Approval freezes a baseline identified by a digest. An ordinary code-generation agent does not have permission to create an approved baseline merely by editing metadata or Markdown.

6.2 Governed implementation cycle

For each change, the coding tool receives the approved baseline, relevant view descriptions, applicable rule IDs, and the current task specification. It plans and implements the feature. Separately controlled CI checks the resulting code and returns rule-linked evidence. A failure should identify the affected rule and permit a repair without requiring architecture authors to re-explain the system.

If the change legitimately requires a prohibited dependency, the agent can submit an **architecture amendment proposal** showing the old rule, proposed change, rationale, affected views, alternative designs, and impact on existing tests. Only after approval is a new baseline issued. Implementation then targets that new baseline.

The invariant is simple: **implementation may change within the approved architecture; the architecture itself changes only through its own governance path**.

7. Reference validator: architecture and interface

The proposed **AAB reference validator** is a deterministic CLI and library that interprets the AAB package independently of the AI coding agent. It is a design target, not software reported as already built.

7.1 Validator pipeline

1. **Parse and validate syntax.** Parse the package and check it against versioned JSON Schema rules; reject invalid identifiers, duplicate IDs, invalid references, and malformed rule definitions. JSON Schema Draft 2020-12 is a practical existing foundation [13].
2. **Resolve and normalise the model.** Build stable element and relationship indexes, resolve relative paths, and detect contradictory or dangling view mappings.
3. **Verify the approved baseline.** Compare the package digest and approval metadata with trusted base-branch or release records. An unapproved change to a blocking rule is not a valid bypass.
4. **Determine coverage.** Match each active constraint to a supported, enabled checker and an eligible set of artifacts. Missing adapters or incomplete source mappings must be reported.
5. **Execute supported checks.** Use narrowly defined adapters for static dependencies, tests, configuration policies, and other supported assertions. Record checker versions and relevant inputs.

6. **Produce evidence.** Emit machine-readable and human-readable results, preserving the baseline digest, source revision, individual findings, uncovered rules, and tool failures.
7. **Apply gate policy.** A trusted CI job applies the profile-specific release or merge rule, including how unsupported or inconclusive results are treated.

7.2 Proposed command surface

```
aab validate architecture/              # Structure and referential integrity
aab check architecture/ --source .      # Supported implementation checks
aab diff --base <approved> --head <new>  # Architecture change impact
aab report --format json                # Portable evidence report
```

These are proposed CLI shapes, not functioning commands. An implementation should expose stable exit codes and define their meaning: success; detected policy or coverage violation; and invalid input/checker infrastructure failure. Failure of the validator must never be represented as a successful check.

7.3 The evidence contract

AAB should use at least four rule outcomes:

Outcome	Interpretation
`conformant`	An applicable check ran successfully within its declared scope
`violated`	A checker produced evidence of an applicable rule violation
`unverified`	Coverage or information was insufficient to decide
`not_applicable`	The rule does not apply to the checked change or target

Infrastructure failures should appear separately as **execution errors**, not be misclassified as architectural violations. Evidence records also need the scope checked, rule ID, checker and version, source revision, baseline digest, timestamps, finding references, and evidence artifacts. Human review may support a decision, but must not be presented as automated proof.

For an L2 governed baseline, the recommended default is **fail closed**: a blocking rule that is violated, unverified when verification is required, or affected by a checker failure prevents automatic approval. Non-blocking advisory rules can produce warnings. AAB should permit explicitly approved exceptions with expiry, justification, owner, and linked amendment/decision record; silent suppressions are prohibited.

7.4 Security and trust boundaries

The validator is part of the architecture-governance control plane and must be treated accordingly. An implementation agent should not be able to change its own approval rules, replace the checker executable, substitute a new baseline digest, or forge CI reports. For external pull requests, untrusted code and policy proposals must be evaluated against trusted base-branch rules under appropriately isolated CI execution. Provenance and build attestation mechanisms such as SLSA provide relevant complementary ideas, but do not themselves prove architecture conformance [15].

8. Profiles and adoption strategy

AAB 0.1 should support two initial profiles, leaving more demanding assurance profiles to future work:

AAB-L1 — Described. The architecture package is structurally valid, names important stakeholders and concerns, has stable model identifiers, records significant ADRs, and defines the intended 4+1 views or explicit justified omissions. It establishes traceable architecture context without pretending that source-code compliance has been checked.

AAB-L2 — Governed. All L1 obligations apply. At least the binding subset of rules is linked to declared verification mechanisms; approved baselines and change authorisation are independently enforced; CI emits evidence with explicit coverage; and uncovered blocking rules prevent automatic merge. The validator may claim *L2 process conformance for the declared rule set*, not complete proof of system architectural correctness.

A future **AAB-L3 — Assured** could define stronger evidence obligations for selected risk classes, but should not be promised until the protocol, validation semantics, and operational threat model have been independently tested.

Adoption should be incremental. Teams can begin by extracting a small set of meaningful decisions and boundaries from existing documents, defining precise source mappings, and adding one checker. AAB should favour a dozen high-value verified constraints over hundreds of aspirational but untestable rules.

9. Related work and differentiation

AAB builds directly on established software architecture research. Kruchten's model provides the stakeholder-oriented multi-view foundation [1]; ISO/IEC/IEEE 42010 defines the architecture-description vocabulary [2]; ISO/IEC 25010 supplies a product-quality framework [3]; and ADRs preserve the context and consequences of decisions [5]. Structurizr and the C4 model demonstrate the usefulness of generating views from version-controlled architectural models [6]. None of these should be replaced by an AAB-specific diagramming ecosystem.

The second family of related work concerns **specification-driven agentic development**. GitHub Spec Kit offers a staged path from specification through planning, tasks, and implementation, including constitution-style project principles [7]. OpenSpec emphasises reviewable proposals and delta specifications organised around individual changes [8]. Kiro provides structured requirements, design, and task artifacts [9]. AGENTS.md supplies a lightweight, agent-readable place for project instructions [16]. These solve important orchestration and context problems, but are not, by themselves, an interoperable architecture-conformance protocol independent of a particular workflow.

The third family concerns **executable controls**. ArchUnit can check Java package/class dependencies and layered constraints [10]. Open Policy Agent's Conftest evaluates structured configuration against policy assertions [11]. Evolutionary architecture literature describes architectural fitness functions that test important characteristics continuously [4]. AAB proposes a standard connection between an architecture contract and such validators, including what to report when checks are incomplete or not supported.

Established approach	Principal contribution	Intended AAB complement
4+1 / ISO 42010 / C4	Architecture communication and models	Typed contracts linked to model elements

Established approach	Principal contribution	Intended AAB complement
ADRs	Decision history and rationale	Stable links from decisions to active rules
Spec Kit / OpenSpec / Kiro	Planning and agentic workflow	Independent, cross-feature architecture baseline
AGENTS.md	Agent-readable instructions	Generated guidance plus trusted enforcement
ArchUnit / Conftest	Specific executable assertions	Standard rule mapping and evidence format
Architectural fitness functions	Continuous measurement of architectural properties	Portable governance and traceability protocol

Recent 2026 preprints also discuss the need for explicit governance in AI-native engineering. Alenezi proposes a Specification Governance Reference Model [17], while Konrad and colleagues describe implicit architectural decisions by AI agents [14]. These are useful conceptual points of comparison, but their publication as preprints should not be mistaken for independent validation of AAB or its effectiveness. This paper makes no quantitative productivity or defect-reduction claim.

Differentiation. AAB's proposed contribution is not any individual view, YAML file, testing library, or review checkpoint. It is the **interoperable contract tying architectural intent, approved baselines, binding constraints, verification scope, and portable evidence across tools**.

10. Implementation roadmap and research validation

A meaningful first release would contain **(i)** a published vocabulary and JSON Schema; **(ii)** a specification conformance test suite with positive and negative examples; **(iii)** a dependency-rule adapter for one language ecosystem; **(iv)** a configuration-policy adapter; **(v)** a baseline comparison and approval gate; and **(vi)** an evidence report format. A Spec Kit or OpenSpec integration can then generate applicable instructions and import task links without becoming a mandatory runtime dependency.

The following milestones offer a disciplined path:

Stage	Deliverable	Exit criterion
M1: Semantics	Core entities, typed constraints, explicit exclusions	Independent reviewers resolve examples consistently
M2: Package	Published schema, canonical digest process, test vectors	Two parsers agree on valid/invalid packages
M3: Validator	CLI, model resolution, one dependency checker	Mutation tests detect planted forbidden imports
M4: Governance	Trusted baseline comparison and amendment handling	Editing a rule in a feature branch cannot bypass the gate
M5: Evidence	Portable JSON report, coverage rules, human-readable report	Repeated runs are comparable and auditable
M6: Evaluation	Multi-repository, multi-agent case study	Benefits, cost, and false positives measured transparently

A controlled evaluation should compare **prompt-only**, **spec-driven without AAB**, and **spec-driven with AAB** conditions on identical architecture-sensitive tasks. Record the number and severity of architectural violations, recall on seeded violations, false-positive rates, percentage of binding rules actually covered, human review effort, implementation time, and frequency of legitimate architecture amendments. Repeat tasks across multiple agents and repositories. Assess compliance by independent reviewers and predefined oracles rather than by the implementing agent's self-report.

The hypothesis is that AAB can reduce undetected architectural drift while improving review traceability; that is a **hypothesis to test**, not a result established by this whitepaper.

11. Limitations and open questions

Semantic precision versus usability. The more expressive a rule language becomes, the harder it is for implementers to support consistently. The initial vocabulary should be intentionally narrow and well tested.

Incomplete observation. Static code analysis cannot establish every runtime property; passing tests may not cover real production conditions. Evidence must state its scope and limits without claiming universal correctness.

Model-code correspondence. Source paths, packages, generated code, dynamic imports, external services, and infrastructure definitions can make mappings ambiguous. Unresolved mappings require explicit diagnostics rather than optimistic interpretation.

Governance overhead. A rigid approval process could obstruct exploratory work or encourage teams to bypass controls. Lightweight profiles, risk-based enforcement, and clear amendment handling are essential.

Trust in the verifier. A validator's output is only as credible as its implementation, configuration, execution environment, and provenance. Independent audit, adversarial testing, and protected CI are part of the design problem.

Interoperability. YAML and JSON are not enough. Tools must agree on identifiers, semantics, path resolution, canonicalisation, rule-result meaning, and version compatibility. A reference validator is necessary but not sufficient; an independent second implementation and interoperability fixtures are the real test.

12. Conclusion

Agentic software engineering makes it increasingly valuable to separate the authority to **define architecture** from the activity of **implementing software**. The objective is not to restrict AI to mechanical coding or to return to heavyweight up-front design. It is to preserve the benefits of rapid iteration while making significant architectural choices deliberate, visible, and governable.

AAB 0.1 proposes an attainable first step: combine human-readable 4+1 descriptions with a versioned machine-readable architecture model, a limited set of explicit contracts, an approved baseline, and independent evidence of conformance. Existing standards, spec-driven workflows, and policy engines provide much of the foundation. The missing work is to standardise the connections between them and to demonstrate that different tools interpret those connections in the same way.

The central promise should be stated precisely. **AAB cannot guarantee that agent-generated software is architecturally correct. It can make architectural intent explicit,**

require controlled changes to that intent, check specified properties reproducibly, and show which claims remain unverified. That is a meaningful and testable advance over architecture guidance embedded only in documents and prompts.

Recommended next step: publish the AAB 0.1 core schema and normative semantics as an open discussion draft, implement a minimal independent validator, and evaluate it on an existing architecture-sensitive codebase before proposing formal standardisation.

References

[1] Kruchten, P. (1995). "The 4+1 View Model of Architecture." *IEEE Software*, 12(6), 42–50. https://doi.org/10.1109/52.469759

[2] ISO/IEC/IEEE (2022). *ISO/IEC/IEEE 42010:2022: Software, systems and enterprise — Architecture description*. https://www.iso.org/standard/74393.html

[3] ISO/IEC (2023). *ISO/IEC 25010:2023: Systems and software engineering — SQuaRE — Product quality model*. https://www.iso.org/standard/78176.html

[4] Ford, N., Parsons, R., & Kua, P. (2017). *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media. https://www.oreilly.com/library/view/building-evolutionary-architectures/9781491986356/

[5] Nygard, M. (2011). "Documenting Architecture Decisions." Cognitect blog, 15 November. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[6] Brown, S. (n.d.). *The C4 Model* and *Structurizr: Models as Code*. https://c4model.com/ and https://docs.structurizr.com/

[7] GitHub (2026). *Spec Kit: Spec-driven development*. Project documentation. https://github.com/github/spec-kit and https://github.github.com/spec-kit/

[8] Fission AI (2026). *OpenSpec: Artifact-guided specifications and change proposals*. Project documentation. https://github.com/Fission-AI/OpenSpec

[9] Kiro (2026). *Kiro Specs documentation*. https://kiro.dev/docs/specs/quick-spec/

[10] ArchUnit contributors (2026). *ArchUnit User Guide*. https://www.archunit.org/userguide/html/000_Index.html

[11] Open Policy Agent contributors (2026). *Conftest: Test structured configuration using Rego*. https://github.com/open-policy-agent/conftest

[12] Bradner, S. (1997). *RFC 2119: Key words for use in RFCs to Indicate Requirement Levels*. IETF. https://www.rfc-editor.org/rfc/rfc2119 ; Leiba, B. (2017). *RFC 8174: Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. IETF. https://www.rfc-editor.org/rfc/rfc8174

[13] JSON Schema contributors (2022). *JSON Schema Draft 2020-12*. https://json-schema.org/draft/2020-12/

[14] Konrad, P. M., Adam, T. L., Terrenzi, R., & Ayvaz, S. (2026). "Architecture Without Architects: How AI Coding Agents Shape Software Architecture." *arXiv preprint* arXiv:2604.04990. https://arxiv.org/abs/2604.04990

[15] SLSA contributors (2026). *SLSA Specification v1.2: Provenance*. https://slsa.dev/spec/v1.2/provenance

[16] AGENTS.md contributors (2026). *AGENTS.md: A simple, open format for guiding coding agents*. https://agents.md/

[17] Alenezi, M. (2026). "Specification-Driven Development as the Foundation of AI-Native Enterprise Software Engineering." *arXiv preprint* arXiv:2607.16680. https://arxiv.org/abs/2607.16680

Source note: references were checked against publisher, standards-body, project-maintainer, or paper records available on 12 October 2026. Dates for continuously maintained software documentation indicate the consultation period rather than an immutable release; the two arXiv works are explicitly treated as preprints.